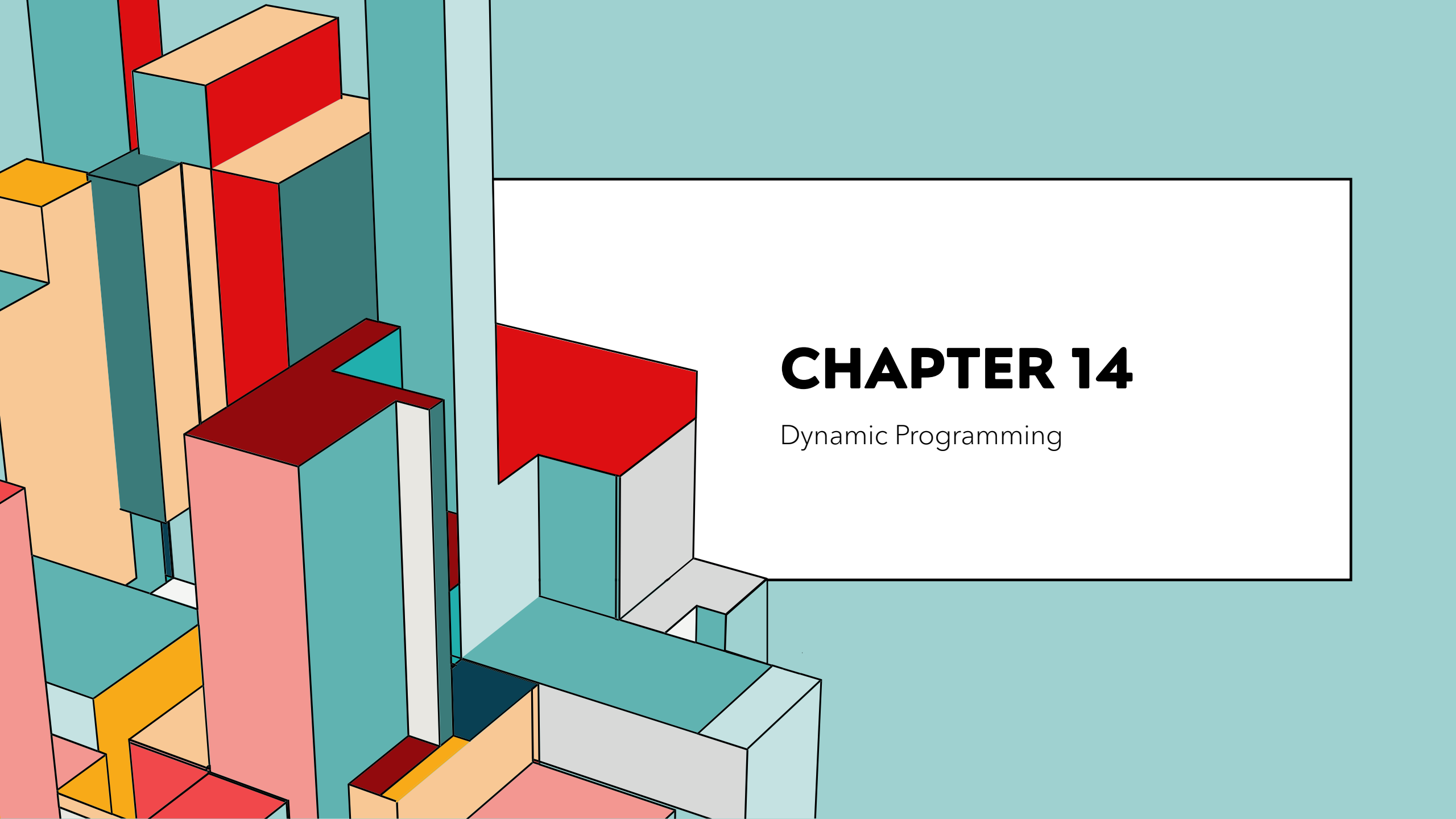




CHAPTER 14

Dynamic Programming

DIVIDE AND CONQUER

- หลักการสำคัญ : แก้ปัญหาโดยการแก้ปัญหาย่อยๆ ซ้ำๆ ไปเรื่อยๆ แล้วนำผลลัพธ์ที่ได้นำมาใช้ในการแก้ปัญหที่ใหญ่ขึ้น
- ประเด็นปัญหา : การทำปัญหาย่อยซ้ำๆ ทำให้โปรแกรมทำงานช้า
- ทำอย่างไรให้ประมวลผลได้เร็ว -> จำค่าผลลัพธ์ที่เคยคำนวณมาแล้ว

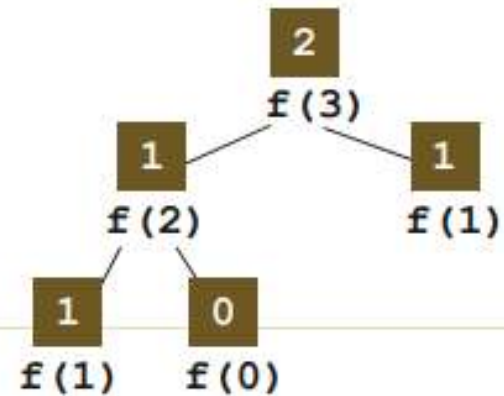
FIBONACCI

$$f_n = f_{n-1} + f_{n-2}$$

$$f_0 = 0, f_1 = 1$$

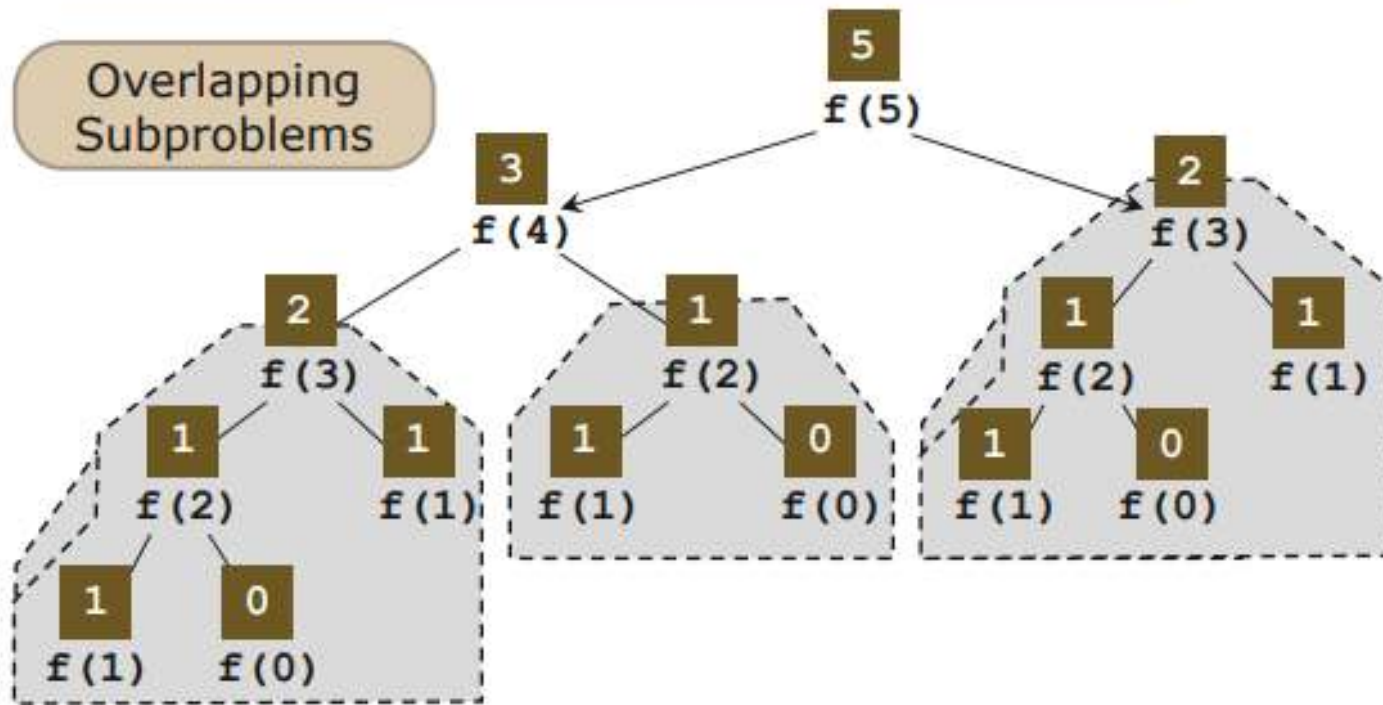
Top-down

```
f( n ) {  
    if (n < 2) return n  
    return f(n - 1) + f(n - 2)  
}
```



FIBONACCI : ทำ F(N) ซ้ำๆ

```
f( n ) {  
    if (n < 2) return n  
    return f(n - 1) + f(n - 2)  
}
```

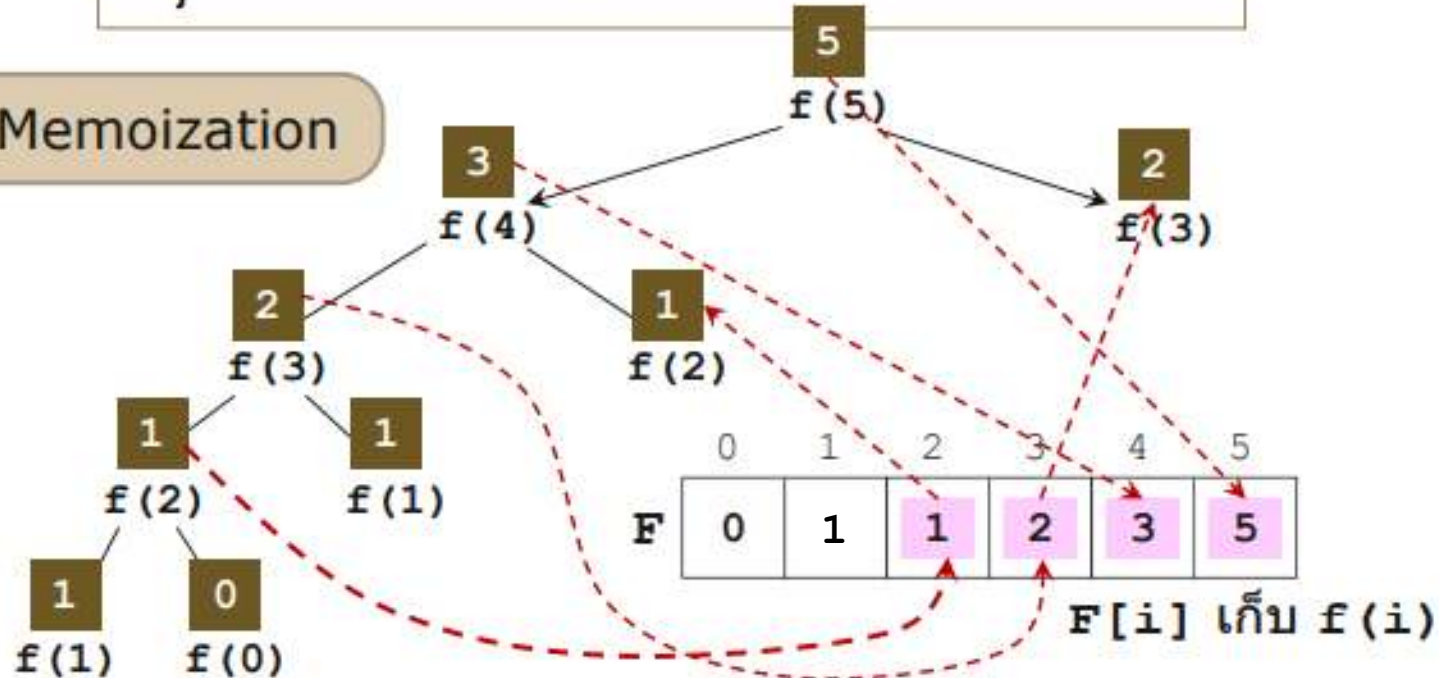


FIBONACCI : จำค่า F(N) ที่เคยคำนวณแล้ว

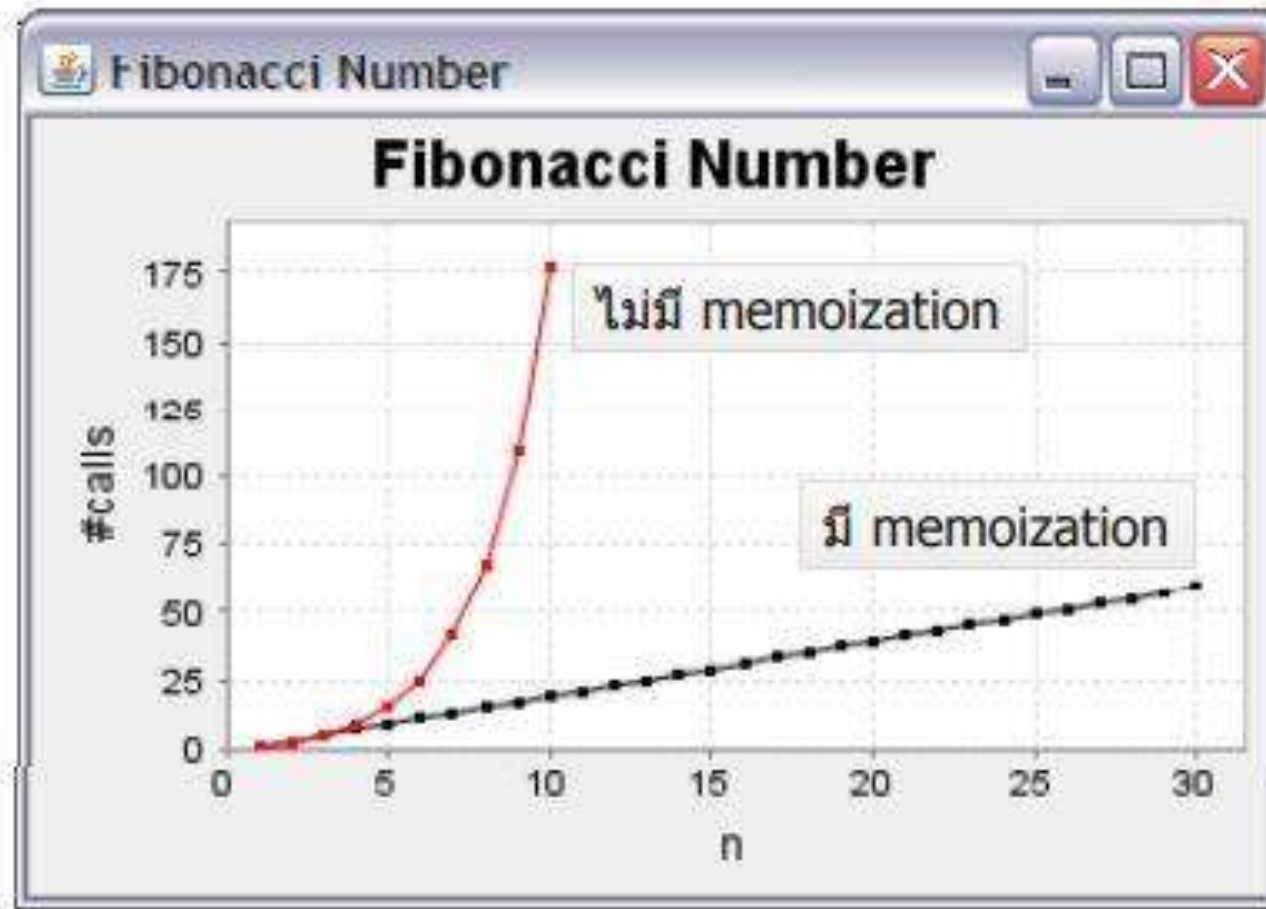
```
f(n, F[0..n]) {  
    if (n < 2) return n  
    if (F[n] > 0) return F[n]  
    return F[n] = f(n-1, F) + f(n-2, F)  
}
```

$\Theta(n)$

Memoization



FIBONACCI



FIBONACCI : BOTTOM-UP

	0	1	2	3	4	5
F	0	1	1	2	3	5

```
f( n ) {  
    F = new array[0..n]  
    F[0] = 0; F[1] = 1  
    for (i = 2; i <= n; i++) {  
        F[i] = F[i - 1] + F[i - 2]  
    }  
    return F[n]  
}
```

$\Theta(n)$

bottom-up

DYNAMIC PROGRAMMING

❖ การทำงานโดยทำการแก้ปัญหาย่อย **จดจำผลลัพธ์** และนำไปใช้แก้ปัญหาที่ใหญ่ขึ้นต่อไป

↖ การหาค่า
❖ มักใช้กับ **Optimization problems**

- เลือกหยิบของที่มีมูลค่ารวมมากที่สุด
- หาข้อความที่เหมือนกันมากที่สุด
- หาเส้นทางสั้นสุด

ปัญหาที่ใช้ DYNAMIC PROGRAMMING

- ❖ Knapsack problem
- ❖ Longest Common Substring
- ❖ Longest Common Subsequence

KNAPSACK PROBLEM

- ❖ ปัญหา : จะหยิบสิ่งของใส่เป้ (หรือตะกร้า) อย่างไร
- ได้ของที่มีมูลค่ารวมมากที่สุด
 - ไม่ให้เกินน้ำหนักที่เป้จะสามารถรับได้ (4 lbs)



STEREO
\$3000
4 lbs



LAPTOP
\$2000
3 lbs



GUITAR
\$1500
1 lbs

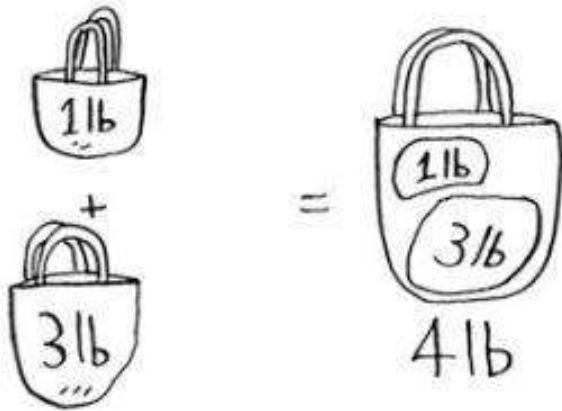
KNAPSACK PROBLEM: SIMPLE SOLUTION

- ทดลองหยิบของทุกกรณีที่เป็นไปได้ทั้งหมดตามเงื่อนไข แล้วเลือกกรณีที่ทำให้มูลค่ารวมของที่หยิบมากที่สุด $\rightarrow O(2^n)$



KNAPSACK: DYNAMIC PROGRAMMING

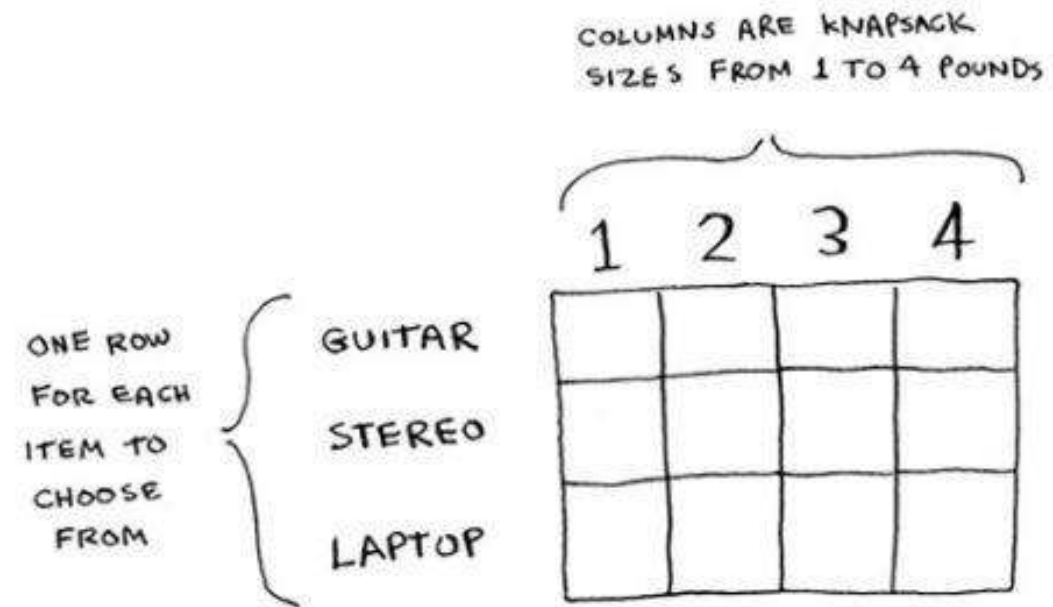
- ❖ พยายามย่อยปัญหาและแก้ไขปัญหาย่อยๆ ก่อน
 - ให้คิดเหมือนเป้ของเราที่มีขนาดเล็กลงหลายใบ และเราพยายามใส่ของในเป้เล็กแต่ละใบให้ได้มูลค่ามากที่สุด
 - การแก้ปัญหาลใหญ่ = การนำผลลัพธ์การแก้ปัญหาย่อยมารวมกัน



KNAPSACK: DYNAMIC PROGRAMMING

❖ แนวคิดในการทำ Dynamic Programming ง่ายๆ จะทำได้ด้วยการใช้ Grid

- Row : สิ่งของต่างๆ
- Column : น้ำหนักที่เป็รับได้



KNAPSACK: DYNAMIC PROGRAMMING

❖ เริ่มเติมค่าเข้าไปใน Grid

g : 1 kg , \$ 1500

- เริ่มที่แถว Guitar -> คิดเหมือนว่า ในร้านมีกีตาร์ให้หยิบอย่างเดียว
- ให้เขียนมูลค่าของทีมากที่สุดที่จะหยิบได้ไม่เกินน้ำหนักเป้ ไล่จากเป้ขนาด 1 ปอนด์ ถึง 4 ปอนด์ (เหมือนแก้ปัญหาย่อยไปใหญ่)

	1	2	3	4
GUITAR	\$1500 G	\$1500 G	\$1500 G	\$1500 G
STEREO				
LAPTOP				

← OUR CURRENT
BEST GUESS
FOR WHAT THE
THIEF SHOULD STEAL:
THE GUITAR
FOR \$1500

KNAPSACK: DYNAMIC PROGRAMMING

G : 1 kg , \$1500

S : 4 kg , \$3000

❖ แอว Stereo

- คิดเหมือนว่า เราสามารถหยิบได้ทั้ง Stereo และ Guitar
- หมายถึง เราสามารถหยิบของจากแถวด้านบนมาได้

	1	2	3	4
GUITAR	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 G
STEREO	\$1500 G	\$1500 G	\$1500 G	\$3000 S
LAPTOP				

KNAPSACK: DYNAMIC PROGRAMMING

❖ แกว Laptop

- คิดเหมือนว่า เราสามารถหยิบได้ทั้ง Laptop, Stereo และ Guitar
- ทำเหมือนเดิม

	1	2	3	4
GUITAR	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
STEREO	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 S
LAPTOP	\$1500 ↓ G	\$1500 ↓ G	\$2000 L	

$$\text{\$3000 STEREO} \text{ vs } \left(\text{\$2000 LAPTOP} + \frac{???}{1 \text{ LB OF FREE SPACE}} \right)$$

	1	2	3	4
GUITAR	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
STEREO	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 S
LAPTOP	\$1500 ↓ G	\$1500 ↓ G	\$2000 L	\$3500 L G

↑
THE ANSWER!

KNAPSACK: DYNAMIC PROGRAMMING

- ❖ คำตอบสุดท้าย -> Guitar + Laptop -> \$3500
- ❖ ถ้าสังเกตดีๆ จะเห็นว่ามีรูปแบบในการคิดซ้ำๆ และสามารถเขียนสูตรในรูปแบบของ Recursion เพื่อเก็บค่าใน Grid ได้ดังนี้

$$\begin{array}{c} \text{ROW} \quad \text{COLUMN} \\ \downarrow \quad \downarrow \\ \text{CELL}[i][j] \end{array} = \max \text{ of } \left\{ \begin{array}{l} 1. \text{ THE PREVIOUS MAX (VALUE AT CELL } [i-1][j]) \\ \text{VS} \\ 2. \text{ VALUE OF CURRENT ITEM + VALUE OF THE REMAINING SPACE} \\ \quad \quad \quad \uparrow \\ \quad \quad \quad \text{CELL}[i-1][j - \text{ITEM'S WEIGHT}] \end{array} \right.$$

KNAPSACK: DYNAMIC PROGRAMMING

- ❖ สมมติว่ามีของในร้านเพิ่มอีก 1 อย่าง คือ iPhone
- ❖ ให้แก้ปัญห Knapsack ของเป้ขนาด 4 ปอนด์ -> จะต้องหยิบสิ่งของใดบ้างที่ทำให้ได้มูลค่าของรวมกันมากที่สุด

	1	2	3	4
GUITAR	\$1500 G	\$1500 G	\$1500 G	\$1500 G
STEREO	\$1500 G	\$1500 G	\$1500 G	\$3000 S
LAPTOP	\$1500 G	\$1500 G	\$2000 L	\$3500 LG
IPHONE	\$2000 1	\$3500 1G	\$3500 1G	\$4000 L1

↑
NEW ANSWER



IPHONE
\$2000
11b

แนวคิด DYNAMIC PROGRAMMING

- ❖ สร้าง Grid เพื่อใช้ในการเก็บผลลัพธ์ย่อยๆ และนำไปแก้ปัญหาลูกใหญ่ได้เร็วขึ้น (ไม่ต้องคำนวณใหม่)
- ❖ ใส่ค่าเข้าไปในแต่ละเซลล์ใน Grid โดยค่าที่ใส่นั้นมักจะเป็นค่าที่สะท้อนถึง ความคุ้มค่า หรือทางที่ดีที่สุดของการแก้ปัญหา
 - ค่าเซลล์ใน Knapsack คือ มูลค่าสินค้าที่ยิบ
- ❖ แต่ละเซลล์ใน Grid จะหมายถึงคำตอบของปัญหาย่อย

LONGEST COMMON SUBSTRING

- ปัญหา : ให้หาระดับความคล้ายคลึงระหว่างคำ 2 คำ
 - ใช้ในกรณีค้นหาคำศัพท์ แต่อาจจะพิมพ์ผิด โปรแกรมสามารถเดาคำศัพท์ที่มีความคล้ายคลึงกับคำที่หาให้ได้
 - HISH & FISH
 - HISH & VISTA
 - สูตรการคำนวณควรเป็นอย่างไร

	H	I	S	H
F	0	0		
I				
S			2	0
H				3

LONGEST COMMON SUBSTRING

```
if word_a[i] == word_b[j] :  
    cell[i][j] = cell[i-1][j-1] + 1  
else:  
    cell[i][j] = 0
```

1. IF THE LETTERS
DON'T MATCH,
THE VALUE IS
ZERO.

	H	I	S	H
F	0	0	0	0
I	0	1	0	0
S	0	0	2	0
H	0	0	0	3

2. IF THEY DO MATCH,
THIS VALUE IS
VALUE OF TOP-LEFT NEIGHBOR + 1

LONGEST COMMON SUBSTRING

- HISH & VISTA
- คำตอบไม่จำเป็นต้องเป็นเซลล์สุดท้าย
- กรณีนี้ คำตอบคือค่าที่มากที่สุดใน Grid
- HISH และ VISTA มีค่าความคล้ายเป็น 2

	V	I	S	T	A
H	0	0	0	0	0
I	0	1	0	0	0
S	0	0	2	0	0
H	0	0	0	0	0

FINAL ANSWER

↑
NOT THE FINAL ANSWER

LONGEST COMMON SUBSEQUENCE

- วิธีหาความคล้ายของคำก่อนหน้านี้ อาจจะทำให้ผลลัพธ์ที่ไม่ดีในบางกรณี เช่น
 - ความคล้ายของ FOSH และ FORT เป็น 2
 - ความคล้ายของ FOSH และ FISH เป็น 2 ☹️

	F	O	S	H
F	1	0	0	0
O	0	2	0	0
R	0	0	0	0
T	0	0	0	0

vs

	F	O	S	H
F	1	0	0	0
I	0	0	0	0
S	0	0	1	0
H	0	0	0	2

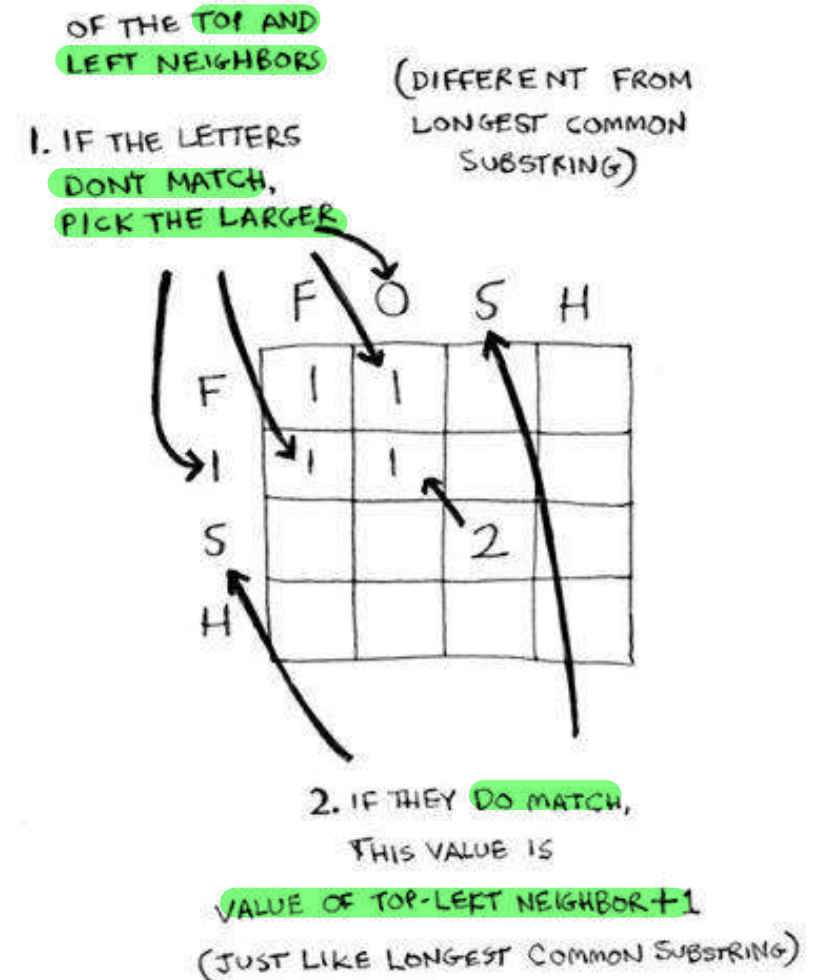
LONGEST COMMON SUBSEQUENCE

- ลองเพิ่มเติมวิธีคิด
 - แทนที่จะหาขนาดข้อความย่อย (Substring) ที่เหมือนกัน
 - พิจารณา Subsequence ที่เหมือนกันมากที่สุดแทน
 - Subsequence ของ FOSH คือ $\langle F, O \rangle$, $\langle F, S \rangle$, $\langle F, H \rangle$, $\langle F, O, S \rangle$, $\langle F, O, H \rangle$, ...

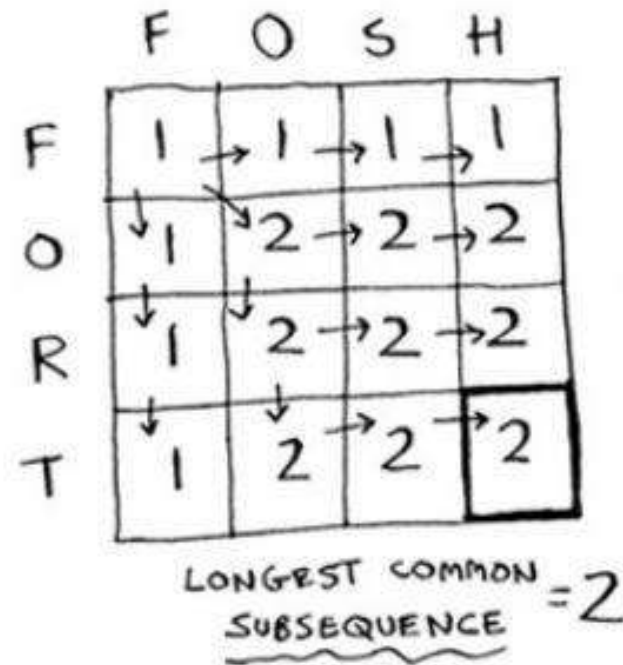
	F	O	S	H
F	1	1		
I	1			
S		1	2	2
H				

LONGEST COMMON SUBSEQUENCE

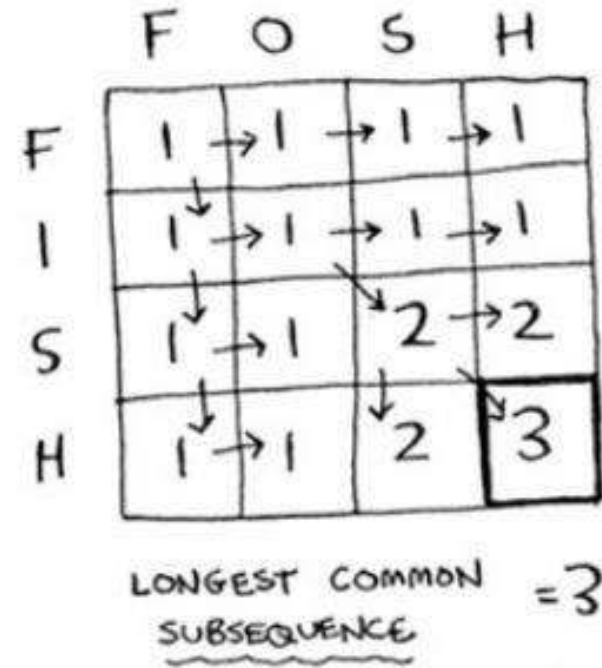
```
if word_a[i] == word_b[j] :  
    cell[i][j] = cell[i-1][j-1] + 1  
else:  
    cell[i][j] = max(cell[i-1][j], cell[i][j-1])
```



LONGEST COMMON SUBSEQUENCE



vs



EXERCISE 1

- ❖ สมมติว่าคุณเดินทางไปประเทศอังกฤษ และมีเวลาท่องเที่ยวเป็นเวลา 2 วัน คุณควรวางแผนจะไปสถานที่ท่องเที่ยวต่างๆ อย่างไร ที่จะได้ผลรวมเรตติ้งของสถานที่ที่ไปมากที่สุด
- ให้เขียนสูตรที่ใช้ในการทำ Dynamic Programming (Recursion)
 - ให้เขียนและเติมค่า Grid ที่ใช้ในการคำนวณ Dynamic Programming

ATTRACTION	TIME	RATING
WESTMINSTER ABBEY	1/2 DAY	7
GLOBE THEATER	1/2 DAY	6
NATIONAL GALLERY	1 DAY	9
BRITISH MUSEUM	2 DAYS	9
ST. PAUL'S CATHEDRAL	1/2 DAY	8

EXERCISE 2-3

- จงหาค่า Longest Common Substring ของคำว่า “together” และ “tighter”
(พร้อมแสดง Grid และค่าใน Grid)
- จงหาค่า Longest Common Subsequence ของคำว่า “together” และ “tighter”
(พร้อมแสดง Grid และค่าใน Grid)